# 30. Channels, Select, and Pipelines

## 30. Channels, Select, and Pipelines

**Phase:** Phase 5 — Concurrency
**Estimated time:** 2 days
**Slug:** `30-channels-select`

### Learning objectives

- Create buffered and unbuffered channels
- Use select for multi-wait
- Build pipeline stages
- Close channels from the sender side only

### Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

### Why this matters

Channels synchronize and communicate. Select lets you wait on multiple events — timeouts, cancellation, and data — in one place.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

### Core concepts

- chan T is typed; make(chan T, n) sets buffer size.
- Send/receive block until the other side is ready (unbuffered).
- close signals no more sends; range over chan until closed.
- select chooses among ready cases; default is non-blocking.
- Prefer ownership: only the sender closes.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

### Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-channels, && cd /tmp/gw-channels,
go mod init example.com/scratch
```

Then create `main.go`:

```
package main
```

```
import "fmt"

func main() {
    ch := make(chan int, 2)
    ch <- 1
    ch <- 2
    close(ch)
    for v := range ch {
        fmt.Println(v)
    }
}
```

Run it:

```
go run .
```

Expected output:

```
1
2
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

## Mini-lab (`go run` steps)

1. Pipeline: generator → square → print with channels.
2. select with time.After for a timeout.
3. Show deadlock with unbuffered send and no receiver.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

## Curriculum notes (expanded from the compact draft)

## Channels

```
ch := make(chan int)      // unbuffered
buf := make(chan int, 16) // buffered
ch <- 3
v := <-ch
close(ch)
```

Unbuffered send/receive synchronizes two goroutines. Buffer decouples up to capacity.

## select

```
select {
case v := <-in:
    // handle
case <-ctx.Done():
    return ctx.Err()
case out <- v:
```

```
    // send
default:
    // non-blocking path
}
```

## Pipelines

Stage functions: `in <-chan T` → `out <-chan U`, close out when done. Receivers use `for v := range ch`.

## Failure modes to watch while coding

- Closing channels from receivers
- Sending on closed channel (panic)
- Unbounded goroutine fan-out without backpressure

When a test fails, ask which failure mode you hit before changing code at random.

## Idioms vs Python pitfalls

- queue.Queue ≈ buffered channel, but channels also sync.
- No select equivalent as clean as Go's.
- Generators vs channel producers — close when done.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would "work". The goal is a Go-shaped intuition.

## Going deeper

Pipelines: each stage closes its output when the input is drained.

## Practice checklist

Before the exercise, confirm you can:

1. Explain "Channels, Select, and Pipelines" aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson's rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

## Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

## Common mistakes (especially from Python instincts)

- Closing channels from receivers
- Sending on closed channel (panic)
- Unbounded goroutine fan-out without backpressure

## Exercises

Work in `exercises/30-channels-select/`. Read that folder's `README.md` before you code.

### What you will build

Wire channel pipeline/select behavior required by tests.

### Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

**Do not open `solutions/30-channels-select/` until you have an honest failing attempt.**

## Self-check

Answer without scrolling up:

1. Who should close a channel?
2. Buffered vs unbuffered?
3. What does range on channel do?

## Further reading

- https://go.dev/blog/pipelines